Robustness of Text Classification to Adversarial Unicode Perturbations: A Comparative Study of Word-, Character-, and Subword-Based Representations

Lorenzo Marinelli (2043092)
Sapienza University of Rome

Academic Year 2025–2026

Abstract

Text classifiers receive Unicode strings, while readers primarily perceive rendered glyphs. This mismatch enables adversarial changes that preserve the apparent text but alter its code points and tokenization. We compare word and character TF–IDF classifiers, an `o200k_base` BPE-feature classifier, and fine-tuned DistilBERT on BANKING77 intent detection and coarse TREC question classification. The evaluation covers homoglyph substitutions, invisible characters, compatibility characters, canonical composition, and mixed attacks at four strengths. We additionally evaluate Unicode normalization, removal of default-ignorable characters, and an English-specific confusable-repair heuristic. Beyond accuracy and Macro-F1, we measure attack success, defense recovery, token-sequence disruption, and the association between tokenization changes and failures. The resulting study separates controlled representation effects from the behavior of a pretrained Transformer and identifies both the value and the limits of low-cost input sanitization.

1 Introduction

Text classification is commonly formulated as a mapping $f(x) = y$ from a Unicode string x to a discrete label y. For an intent detector, "How can I reset my PIN?" should map to a password or PIN-related intent. A user can, however, replace Latin `a` (U+0061) with Cyrillic small letter a (U+0430), insert a zero-width character, use a compatibility form, or change canonical composition. The rendered utterance can remain indistinguishable or nearly indistinguishable while the classifier receives a different sequence of code points.

This is a practical concern for intent classification, spam and phishing filters, content moderation, and other systems exposed to untrusted text. It is also a useful scientific setting: word, character, and subword representations transform the same perturbation in different ways. A word vocabulary may turn one modified character into an out-of-vocabulary token; character n-grams may lose only local features; and byte-pair or WordPiece tokenizers may fragment one familiar word into several uncommon tokens.

We address four questions:

1. How does classification degrade as the perturbation rate increases?
2. Are word-, character-, BPE-, and Transformer-based systems affected differently?
3. Are subword tokenization changes associated with attack success?
4. Can lightweight Unicode preprocessing recover attacked performance without harming clean inputs?

The central comparison holds the downstream classical classifier fixed: three TF–IDF representations feed Logistic Regression. DistilBERT is included as a modern pretrained reference rather than as a controlled representation-only comparison. All models receive exactly the same attacked examples.

The contribution is a reproducible experiment rather than a new attack algorithm. It combines (i) a shared, auditable attack corpus with gradual budgets and three attack seeds; (ii) a controlled comparison of word, character, and BPE features; (iii) a pretrained Transformer reference with three fine-tuning seeds; (iv) deployment-style defenses applied to both clean and attacked inputs; and (v) paired uncertainty and tokenization diagnostics. This design makes it possible to distinguish an attack's nominal strength from its effective coverage and to ask whether a defense restores performance at the cost of changing legitimate clean text.

2 Related Work

Eger et al. (2019) show that visually similar character substitutions substantially damage neural and non-neural NLP systems across character-, word-, and sentence-level tasks. They also study visual embeddings, adversarial training, and rule-based recovery. Boucher et al. (2022) broaden the threat model to imperceptible encoding-level attacks, including invisible characters, homoglyphs, reorderings, and deletion controls, and demonstrate black-box failures in deployed and open-source systems.

Both studies motivate a representation-level comparison, but their scope is broader than the present controlled classification setting. Eger et al. provide VIPER attack and shielding code, while Boucher et al. provide generators and detectors for encoding perturbations. Here, those ideas are narrowed to auditable, single-code-point transformations with fixed budgets so every classifier can be evaluated on identical rows.

Tokenization provides a complementary explanation. Chai et al. (2024) analyze subword sensitivity to typographical and formatting variation and show that larger models do not eliminate the problem. Geh et al. (2025) manipulate the selection of valid token sequences without changing the visible input. Their attack is not a Unicode perturbation, but it establishes tokenization itself as a security-relevant interface.

Chai et al. release their TKEval code and data and report that regularized tokenization can improve resilience. Geh et al. instead exploit alternative valid segmentations at the model interface; this is adjacent tokenization-security work, not evidence about Unicode homoglyphs. Our token edit, count, and overlap measurements connect encoding perturbations to failures without conflating these distinct threat models.

For defenses, Cooper et al. (2025) use proprietary and open-source language models to normalize homoglyph-obfuscated text. Our focus is deliberately cheaper: deterministic normalization, removal of default-ignorable characters, and a restricted confusable map. Unicode normalization has precise equivalence guarantees, but NFKC can erase meaningful compatibility distinctions (Unicode Consortium, 2025a). Likewise, confusable skeletons are intended for security analysis rather than arbitrary display-text replacement (Unicode Consortium, 2025b). We therefore measure clean-text harm and present confusable repair as an English-specific heuristic.

Cooper et al. release code and an augmented tweet dataset, enabling comparison with more flexible but more expensive normalization. Our experiment does not claim state of the art across all adversarial text defenses. Instead, it establishes transparent low-cost baselines whose transformations can be inspected code point by code point and whose clean-input behavior can be measured directly.

3 Experimental Method

3.1 Datasets

BANKING77 contains 13,083 English banking queries across 77 fine-grained intents (Casanueva et al., 2020). TREC contains 5,952 questions; we use its six coarse classes: abbreviation, entity, description, human, location, and numeric value (Li and Roth, 2002). Official test sets are retained. Ten percent of each official training set is selected with a fixed stratified split for validation.

The final BANKING77 partitions contain 9,002 training, 1,001 validation, and 3,080 test examples. TREC contains 4,897 training, 545 validation, and 500 test examples after removing ten exact questions from the training side that also occur in the published test set. This preserves the official test set while preventing train–test text leakage. Stable example identifiers are assigned before attack generation. Dataset source revisions and cryptographic file hashes are stored with the artifacts; raw data are downloaded on demand and are not redistributed.

3.2 Models

The word system uses word unigrams and bigrams; the character system uses character 3–5-grams. The BPE system tokenizes with OpenAI's open-source `o200k_base` encoding and uses token-ID unigrams and bigrams. Each representation is weighted by TF–IDF and passed to Logistic Regression. The regularization value $C \in \{0.1, 1, 10\}$ is selected by validation Macro-F1.

No model strips accents or normalizes Unicode internally in the controlled comparison. Thus, preprocessing is evaluated explicitly as a defense rather than silently built into a feature extractor. The three classical systems use the same split, classifier family, random seed, and hyperparameter grid; all selected $C = 10$. This does not make their feature spaces equal, but it isolates representation more closely than comparing unrelated classifiers.

DistilBERT (Sanh et al., 2019) is fine-tuned separately on each dataset for three epochs and three random seeds. It uses a maximum sequence length of 128, learning rate 2×10^{-5}, and effective batch size 16. Validation Macro-F1 selects the checkpoint. Results are averaged across fine-tuning seeds.

Fine-tuning seeds are 13, 37, and 73, with weight decay 0.01. Training and inference use Apple's MPS backend. The standard per-device batch is 16; the notebook contains a deterministic fallback to batch 8 with two-step gradient accumulation if memory is exhausted. No paid API or proprietary inference model is used.

3.3 Attacks

Each attack modifies a fraction $s \in \{.05, .10, .20, .40\}$ of eligible positions with three fixed attack seeds. Homoglyphs are restricted to a curated set of single-code-point Greek and Cyrillic lookalikes. Invisible attacks insert default-ignorable characters after alphanumeric positions. Compatibility attacks use characters whose NFKC form recovers the ASCII source. Composition attacks replace eligible precomposed characters with canonically equivalent decompositions. Mixed attacks sample an applicable transformation at each selected position.

For a nonzero s, at least one change is made whenever an eligible position exists. Each output records the original and replacement code points, Unicode names, eligible-position count, and effective change count. Because canonical composition is uncommon in these English datasets, we report its dataset-wide result and its

result restricted to eligible inputs.

Every family is generated at attack seeds 13, 37, and 73, producing 214,800 attacked rows shared by all classifiers. Table 1 shows that four families cover every test example and closely match the requested 20% budget. Neither corpus contains a canonically decomposable test character under the declared rule, so composition has zero coverage: its full-test score equals the clean score and an eligible-subset estimate is undefined rather than treated as evidence of robustness.

3.4 Defenses and Metrics

The five test-time wrappers are NFC, NFKC, removal of default-ignorable characters, NFKC followed by removal, and that combined pipeline followed by restricted confusable repair. Models are not retrained. Clean inputs pass through the same wrapper to expose collateral changes.

We report accuracy, Macro-F1, and robustness drop. Attack success rate (ASR) is the fraction of examples correct when clean but incorrect after attack. For score M, defense recovery is

$$R = \frac{M_{\text{def,att}} - M_{\text{raw,att}}}{M_{\text{raw,clean}} - M_{\text{raw,att}}}. \tag{1}$$

It is not clipped, and is undefined for a nonpositive denominator. Results include variation across attack and model seeds and paired bootstrap intervals over example identifiers.

Accuracy and Macro-F1 are computed for every dataset, model seed, attack family, strength, attack seed, and defense. Means and standard deviations summarize attack and fine-tuning seeds. Paired 95% bootstrap intervals use 500 resamples of aligned example identifiers at the primary 20% strength. This preserves the clean–attacked pairing when estimating score drops. For tokenization, normalized Levenshtein distance compares token-ID sequences, relative count change measures fragmentation, and multiset Jaccard overlap measures retained token content.

4 Classification Robustness

4.1 Clean Performance

Table 2 establishes the utility baseline. Character TF–IDF is strongest on the fine-grained BANKING77 task (0.906 Macro-F1), 0.020 above the word model and 0.024 above BPE features. DistilBERT reaches only 0.819 ± 0.004 on this split despite fine-tuning, illustrating why it is treated as an external reference rather than an assumed upper bound. On coarse TREC, DistilBERT is strongest at 0.944 ± 0.009; the three classical systems cluster between 0.879 and 0.885. Clean performance alone therefore does not predict the most useful model for both datasets.

4.2 Degradation with Strength

Figure 1 shows monotonic degradation for every broadly applicable family. Averaging homoglyph, invisible, compatibility, and mixed attacks, BANKING77 word Macro-F1 falls from 0.780 at 5% strength to 0.205 at 40%; BPE falls from 0.789 to 0.231. Character features degrade more gradually, from 0.886 to 0.653. DistilBERT lies between these regimes, moving from 0.711 to 0.359. The same ordering is visible on TREC: at 40%, character TF–IDF retains 0.603 Macro-F1 and DistilBERT 0.545, compared with 0.319 for word and 0.249 for BPE features.

The largest broadly applicable mean drop is 0.767: BANKING77 DistilBERT under the 40% compatibility attack.

Table 3 summarizes the primary 20% setting. Character TF–IDF has the smallest average drop on both datasets (0.099 on BANKING77 and 0.124 on TREC), whereas BPE and word features lose 0.35–0.43 Macro-F1. DistilBERT is more robust than BPE on TREC but not uniformly on BANKING77. The attack-success rates lead to the same conclusion: character features fail on 12.5% and 17.9% of clean-correct BANKING77 and TREC examples, respectively, while the word model reaches 49.0% and 38.6%.

4.3 Attack-Family Effects

Compatibility characters are the most damaging individual family at 20% for most systems. On BANKING77 they reduce DistilBERT from 0.819 to 0.199 Macro-F1 (ASR 0.777), BPE to 0.410, word features to 0.381, and character features to 0.760. Homoglyphs are also severe, especially for DistilBERT (0.367) and word features (0.574). Invisible characters expose an architectural contrast: they leave DistilBERT predictions unchanged on both datasets because its preprocessing discards the selected controls, but reduce BANKING77 word and BPE Macro-F1 to 0.468 and 0.496. Character n-grams remain local enough to retain 0.830.

The mixed attack does not simply equal the worst family because each selected position samples among applicable transformations. At 20%, its BANKING77 drops are 0.444 (word), 0.411 (BPE), 0.212 (DistilBERT), and 0.095 (character). For one representative attack seed, paired 95% Macro-F1-drop intervals are [0.424, 0.461] for word, [0.395, 0.431] for BPE, [0.082, 0.106] for character, and [0.188, 0.218] for DistilBERT seed 13. On the smaller TREC test set the corresponding intervals are wider, as expected.

4.4 Defense Recovery

The deterministic defenses behave selectively. NFKC reverses compatibility forms but does not repair cross-script homoglyphs or remove controls; ignorable removal reverses the selected invisible insertions but cannot repair compatibility forms. Their combination therefore

Table 1: Attack coverage at the primary 20% strength. "Changes" is the mean number of effective code-point modifications per example.

Attack	B77 cov.	B77 changes	TREC cov.	TREC changes
homoglyph	1.00	4.86	1.00	3.26
invisible	1.00	8.54	1.00	5.64
compatibility	1.00	8.85	1.00	5.88
composition	0.00	0.00	0.00	0.00
mixed	1.00	8.85	1.00	5.88

Table 2: Clean test performance. Transformer rows summarize three fine-tuning seeds.

Dataset	Model	Accuracy	Macro-F1
BANKING77	o200k BPE TF-IDF	0.882	0.882
BANKING77	Character TF-IDF	0.906	0.906
BANKING77	DistilBERT	0.832 ± 0.003	0.819 ± 0.004
BANKING77	Word TF-IDF	0.885	0.886
TREC	o200k BPE TF-IDF	0.890	0.879
TREC	Character TF-IDF	0.888	0.883
TREC	DistilBERT	0.967 ± 0.003	0.944 ± 0.009
TREC	Word TF-IDF	0.888	0.885

recovers more broadly: at 20%, mean Macro-F1 recovery ranges from 0.52 to 0.72 across datasets and models (Table 4). For example, BANKING77 word Macro-F1 rises from 0.466 raw to 0.796 after NFKC plus ignorable removal, while character features rise from 0.807 to 0.884.

Adding the English-specific confusable map restores the clean score for every generated broad family, giving recovery 1.0 in this closed threat model. This perfect result should not be generalized beyond the experiment: the attack and defense share a deliberately restricted reversible whitelist, clean inputs are almost entirely ASCII, and unseen or context-dependent confusables may not be repaired. The clean Macro-F1 delta is 0.000 for every wrapper on both test sets, but a multilingual or symbol-sensitive corpus could show harm.

5 Tokenization and Failure

For both `o200k_base` and DistilBERT WordPiece, we compare clean and attacked token sequences using relative token-count change, normalized token edit distance, and multiset Jaccard overlap. The principal association model is a binomial regression restricted to clean-correct, effectively changed examples:

$$\operatorname{logit} P(\text{failure}) = \beta_0 + \beta_1 d_{\text{tok}} + \beta_2 \Delta n + \mathbf{c}, \qquad (2)$$

where $\mathbf{c}$ controls for dataset, attack family, strength, and model seed when applicable. Standard errors are clustered by original example because each input appears under repeated perturbations. Coefficients are interpreted as associations, not causal effects.

The association is large for both subword systems. For BPE features, a one-standard-deviation increase in normalized token edit distance corresponds to 2.64 times the odds of failure (95% CI 2.46–2.82). For DistilBERT WordPiece the odds ratio is 3.61 (3.38–3.85). Relative token-count change remains positively associated with BPE failure (OR 1.25, 1.19–1.32) after controlling for edit distance, but its DistilBERT coefficient is slightly negative (OR 0.94, 0.91–0.97). This sign difference cautions against using token count alone as a robustness proxy: the identity and ordering of replacement tokens contain information not captured by length.

Figure 3 provides a nonparametric view. Failure rates rise across token-edit quartiles for both datasets and systems, consistent with the regression. The analysis is restricted to clean-correct examples with at least one effective code-point change, so it models attack failure rather than conflating it with baseline errors. Nevertheless, the result is associational. Attack families jointly determine both token disruption and difficulty, and the controls cannot establish that tokenization changes themselves cause each error.

6 Qualitative Error Analysis

The analysis selects cases using declared rules rather than hand-picking: the largest gold-label probability loss among successful attacks, the largest probability recovery after preprocessing, and high-disruption attacks that do not change the prediction. Tables show both rendered text and escaped code points because invisible modifications cannot be audited visually.

Table 3: Raw performance at 20%, averaged over the four broadly applicable attack families and attack/model seeds. Drop is clean minus attacked Macro-F1; ASR conditions on clean-correct examples.

Dataset	Model	Macro-F1	Drop	ASR
BANKING77	o200k BPE TF-IDF	0.496	0.386	0.453
BANKING77	Character TF-IDF	0.807	0.099	0.125
BANKING77	DistilBERT	0.498	0.321	0.408
BANKING77	Word TF-IDF	0.466	0.420	0.490
TREC	o200k BPE TF-IDF	0.447	0.432	0.454
TREC	Character TF-IDF	0.759	0.124	0.179
TREC	DistilBERT	0.701	0.243	0.274
TREC	Word TF-IDF	0.537	0.347	0.386

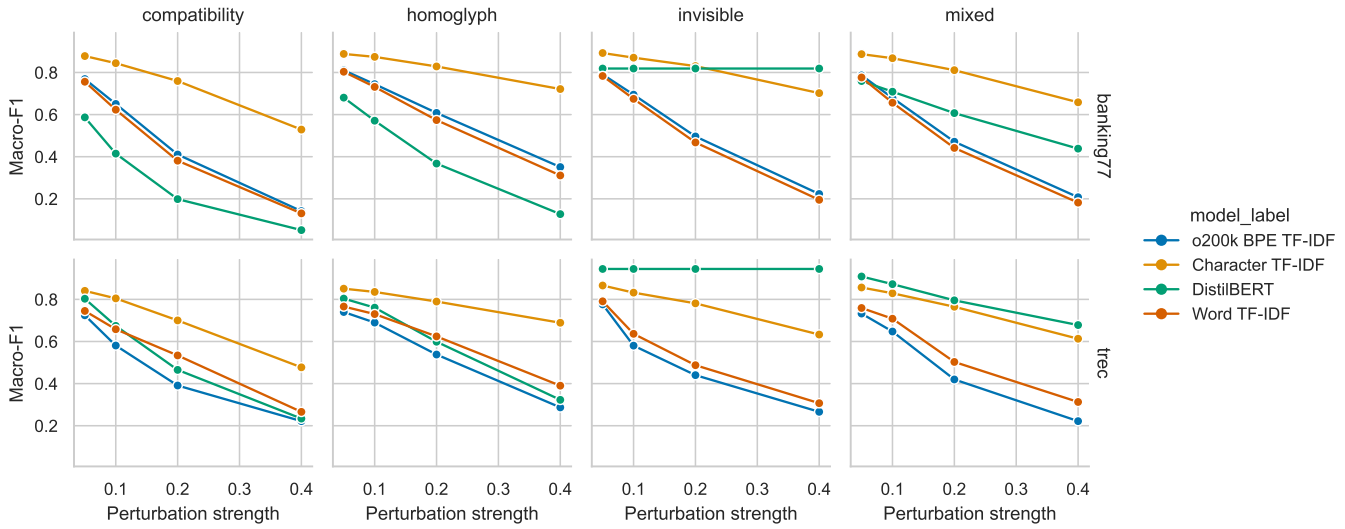


Figure 1: Macro-F1 under increasing perturbation strength. Lines aggregate attack and model seeds; shaded intervals show uncertainty.

The successful BANKING77 example replaces letters in "There is a suspicious direct debit payment" with Cyrillic or related lookalikes. The word vocabulary loses several lexical cues and predicts the wrong intent; the restricted confusable repair reconstructs the clean ASCII string and restores the prediction. By contrast, the invisible attack on "Do you offer Visa or Mastercard?" inserts five default-ignorable controls but does not overturn the selected model prediction. Such resisted cases matter: attack strength counts code-point edits, not guaranteed semantic-feature removal, and redundant class cues can survive substantial local disruption.

The same rendered sentence can therefore represent three distinct computational cases: a failed raw classifier, a repaired classifier, or a classifier that was robust without preprocessing. Reporting only successful examples would exaggerate vulnerability, while reporting only aggregate scores would hide why recovery occurs. The modification log and escaped display make both outcomes reproducible even when the terminal, PDF viewer, or font does not visibly render the inserted characters.

7 Discussion and Limitations

The controlled comparison distinguishes representation effects, but it does not make the systems identical: vocabulary sizes and feature geometries differ, while DistilBERT additionally benefits from pretraining and nonlinear contextualization. Robustness should therefore be compared jointly with clean performance and computational cost.

The clearest representation result is the relative stability of character 3–5-grams. A changed code point destroys only the n-grams that overlap its local neighborhood, while unchanged regions continue to contribute features. Word and BPE representations often replace an entire familiar unit with unseen or rare units. Character robustness is not immunity: its 40% average drop reaches 0.253 on BANKING77 and 0.280 on TREC, and compatibility attacks remain particularly damaging.

DistilBERT demonstrates that a pretrained Transformer is not uniformly robust. Its WordPiece pipeline happens to ignore the selected invisible controls, yet compatibility and homoglyph attacks cause severe fragmen-

Table 4: Macro-F1 at 20%, averaged over four broad families. R is recovery for NFKC plus ignorable removal. "Full repair" additionally maps restricted non-ASCII confusables to ASCII.

Dataset	Model	Raw	NFKC+strip	R	Full repair
BANKING77	o200k BPE TF-IDF	0.496	0.803	0.72	0.882
BANKING77	Character TF-IDF	0.807	0.884	0.72	0.906
BANKING77	DistilBERT	0.498	0.686	0.54	0.819
BANKING77	Word TF-IDF	0.466	0.796	0.72	0.886
TREC	o200k BPE TF-IDF	0.447	0.775	0.71	0.879
TREC	Character TF-IDF	0.759	0.855	0.71	0.883
TREC	DistilBERT	0.701	0.842	0.52	0.944
TREC	Word TF-IDF	0.537	0.808	0.72	0.885

Table 5: Rule-selected examples; non-ASCII and invisible code points are shown explicitly.

Case	Dataset	Model	Attack	Auditable text
Successful attack	BANKING77	Word TF-IDF	homoglyph	clean: There is a suspicious direct debit payment.; attacked: T[U+04BB]ere is a suspi[U+0441][U+0456]ous dire[U+0441]t deb[U+0456]t payment.
Recovered by defense	BANKING77	Word TF-IDF	homoglyph	clean: There is a suspicious direct debit payment.; attacked: T[U+04BB]ere is a suspi[U+0441][U+0456]ous dire[U+0441]t deb[U+0456]t payment.
Prediction resisted	BANKING77	Word TF-IDF	invisible	clean: Do you offer Visa or Mastercard?; attacked: Do yo[U+200B]u of[U+200B]fer Vi[U+200D]s[U+2060]a or Mastercar[U+200B]d?

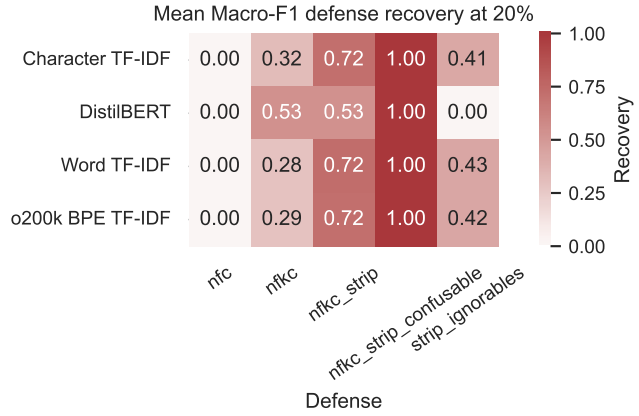


Figure 2: Defense recovery at the primary 20% perturbation strength.

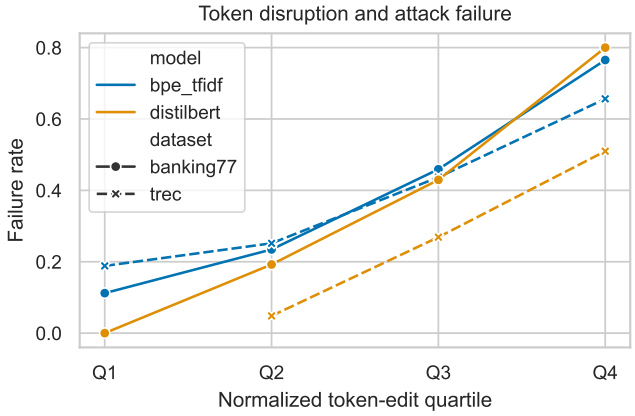


Figure 3: Failure rate by token-sequence disruption quartile.

tation and failure. This family dependence would be obscured by one aggregate Unicode-robustness score. It also explains why defense selection should follow the threat model: NFKC is appropriate for compatibility variants, control removal for default-ignorables, and confusable repair only when the application can justify an English/ASCII assumption.

The attacks approximate visual equivalence through curated mappings and Unicode properties; they do not constitute a human perceptual study. Glyph similarity varies across fonts and platforms. The compatibility attack can produce width or style changes that are noticeable, and the composition attack has low natural coverage in mostly ASCII English text. We publish attack coverage and code-point logs rather than treating every nominal condition as equally effective.

The zero composition coverage is itself an important boundary condition. The study cannot estimate robustness to canonical decomposition on these particular test sets, so the unchanged full-test score must not be reported as a successful defense or robust model. A corpus containing accented Latin text would be needed for that comparison.

The confusable defense is intentionally restricted to English and ASCII targets. Applying it to multilingual input could destroy legitimate distinctions. NFKC likewise folds compatibility distinctions that may carry meaning in other domains. Clean-score deltas are therefore part of the primary evaluation rather than an afterthought.

Finally, repeated attacks on the same test instances create dependent observations. Seed variation and clustered or paired uncertainty estimates reduce, but do not remove, the limits of drawing broad conclusions from two short-text English datasets. Future work could add multilingual corpora, human similarity judgments, font-aware attacks, and adversarial training.

8 Conclusion

This project provides a reproducible test bed for studying how Unicode encoding choices propagate through word, character, BPE, and WordPiece representations. Performance falls monotonically as effective perturbation increases; character n-grams are the strongest lightweight representation; token-sequence disruption is strongly associated with BPE and WordPiece failures; and targeted normalization can recover performance without clean harm on these ASCII-heavy datasets. The perfect full-repair result is a closed-whitelist finding, not evidence for a universal normalizer. Together, gradual budgets, shared perturbed inputs, low-cost defenses, and tokenization diagnostics explain not only whether a classifier fails, but also how the representation contributes to that failure.

9 Statement on the Use of Generative AI

ChatGPT and Codex were used to brainstorm and refine the project design; locate and cross-check primary literature, dataset cards, Unicode standards, and software documentation; scaffold and review the Jupyter notebooks and LaTeX report; assist with debugging, plotting, statistical checks, and proofreading; and document reproducibility steps. No paid model API was used in the experiments. The student remained responsible for running the experiments, checking citations against primary sources, inspecting generated examples and results, evaluating whether interpretations were supported, and approving the final submitted report and code.

References

Nicholas Boucher, Ilia Shumailov, Ross Anderson, and Nicolas Papernot. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy*, pages 1987–2004, 2022. doi: 10.1109/SP46214.2022.9833641.

Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. Efficient intent detection with dual sentence encoders. In *Proceedings of the 2nd Workshop on NLP for Conversational AI*, pages 38–45, 2020. doi: 10.18653/v1/2020.nlp4convai-1.5.

Yekun Chai, Yewei Fang, Qiwei Peng, and Xuhong Li. Tokenization falling short: On subword robustness in large language models. In *Findings of EMNLP*, pages 1582–1599, 2024. doi: 10.18653/v1/2024.findings-emnlp.86.

Portia Cooper, Eduardo Blanco, and Mihai Surdeanu. The lies characters tell: Utilizing large language models to normalize adversarial unicode perturbations. In *Findings of ACL*, pages 18932–18944, 2025. doi: 10.18653/v1/2025.findings-acl.969.

Steffen Eger, Gözde Gül Şahin, Andreas Rücklé, Ji-Ung Lee, Claudia Schulz, Mohsen Mesgar, Krishnkant Swarnkar, Edwin Simpson, and Iryna Gurevych. Text processing like humans do: Visually attacking and shielding nlp systems. In *Proceedings of NAACL-HLT*, pages 1634–1647, 2019. doi: 10.18653/v1/N19-1165.

Renato Lui Geh, Zilei Shao, and Guy Van den Broeck. Adversarial tokenization, 2025.

Xin Li and Dan Roth. Learning question classifiers. In *COLING 2002: The 19th International Conference on Computational Linguistics*, 2002. URL `https://aclanthology.org/C02-1150/`.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

Unicode Consortium. Unicode standard annex #15: Unicode normalization forms, 2025a. URL `https://unicode.org/reports/tr15/`.

Unicode Consortium. Unicode technical standard #39: Unicode security mechanisms, 2025b. URL `https://unicode.org/reports/tr39/`.